

Name :GOPIKA P

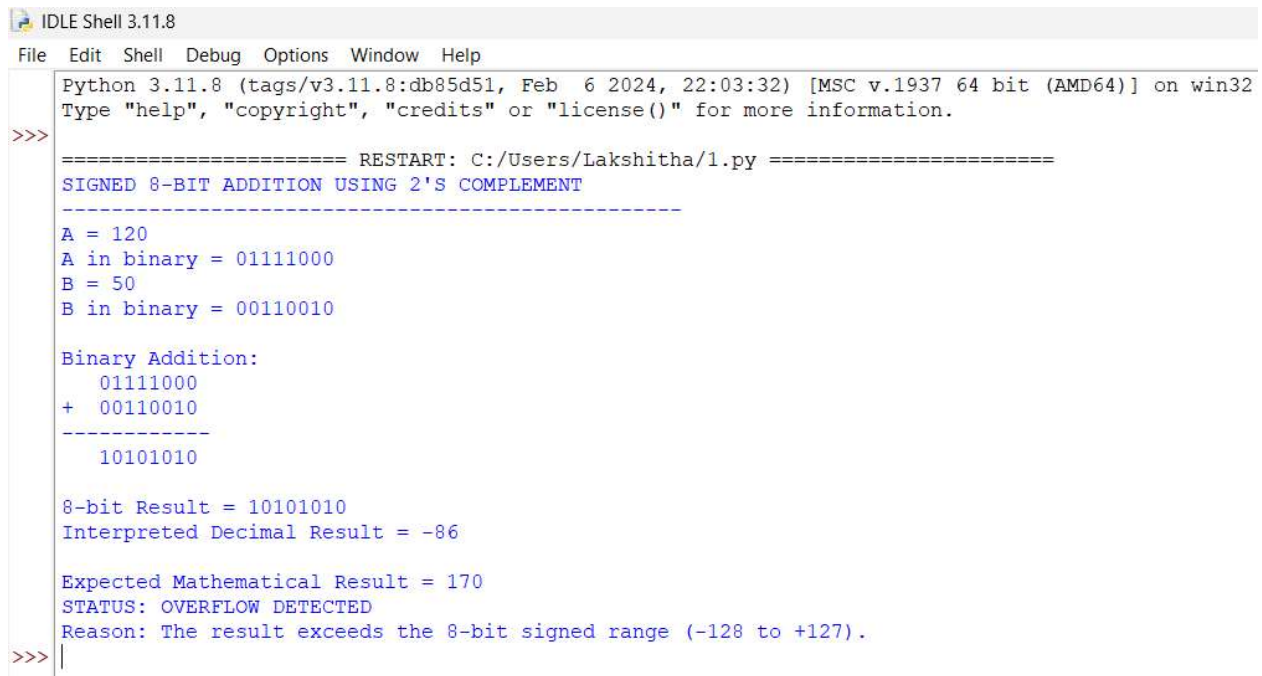
Reg no:192412306

Code :CSA1208

Course : computer architecture

CO2 AT2

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.



```
IDLE Shell 3.11.8
File Edit Shell Debug Options Window Help
Python 3.11.8 (tags/v3.11.8:db85d51, Feb 6 2024, 22:03:32) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Lakshitha/1.py =====
SIGNED 8-BIT ADDITION USING 2'S COMPLEMENT
-----
A = 120
A in binary = 01111000
B = 50
B in binary = 00110010

Binary Addition:
  01111000
+ 00110010
-----
  10101010

8-bit Result = 10101010
Interpreted Decimal Result = -86

Expected Mathematical Result = 170
STATUS: OVERFLOW DETECTED
Reason: The result exceeds the 8-bit signed range (-128 to +127).
>>> |
```

The program adds +120 and +50 using 8-bit 2's complement representation. The binary result is 10101010, which is interpreted as -86 in signed 8-bit representation. However, the actual mathematical result is +170, which is outside the range of -128 to +127. Therefore, overflow occurs. The debugging solution checks the sign of the operands and result and correctly detects the overflow.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

4-BIT RIPPLE CARRY ADDER vs CARRY LOOK-AHEAD ADDER

INPUTS

A = 1011

B = 0110

RIPPLE CARRY ADDER

A = 1011

B = 0110

Result = 10001

Carry propagation = [0, 1, 1, 1]

CARRY LOOK-AHEAD ADDER

Generate signals G:

G3 G2 G1 G0 = 0010

Propagate signals P:

P3 P2 P1 P0 = 1101

Carry signals:

C0 C1 C2 C3 C4 = [0, 0, 1, 1, 1]

CLA Result = 10001

COMPARISON

Ripple Carry : Carry moves from one bit to the next.

CLA : Carries are calculated in advance.

Optimization : CLA reduces carry propagation delay.

FINAL RESULT

1011 + 0110 = 10001

The ripple carry adder waits for the carry from each previous bit, which increases delay. The CLA calculates generate and propagate signals and determines the carries in advance. Therefore, CLA produces the same result with lower carry propagation delay and is more suitable for high-speed processors.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

```
===== RESTART:
BOOTH'S MULTIPLICATION
=====
Multiplicand M = -6
M in binary    = 1010
Multiplier Q   = 5
Q in binary    = 0101
-M in binary   = 0110

Initial Registers
A   = 0000
Q   = 0101
Q-1 = 0

BOOTH ALGORITHM TABLE
-----
SC  Q0 Q-1  Operation          A          Q          Q-1
4   10      A = A - M        0110        0101        0
      Arithmetic Right Shift0011        0010        1
3   01      A = A + M        1101        0010        1
      Arithmetic Right Shift1110        1001        0
2   10      A = A - M        0100        1001        0
      Arithmetic Right Shift0010        0100        1
1   01      A = A + M        1100        0100        1
      Arithmetic Right Shift1110        0010        0

FINAL RESULT
-----
A = 1110
Q = 0010
Product = 11100010
Decimal Result = -30
STATUS: CORRECT
```

The possible bugs in Booth's algorithm are:

1. Wrong Q_0 and Q_{-1} checking
 - 01 →
 - 10 →
 - 00 and 11 → No operation

2. Incorrect shifting
 - Booth requires an arithmetic right shift.
 - The sign bit of A must be preserved.
3. Incorrect sign extension
 - Negative values must maintain their leading 1s during the arithmetic shift.
4. Incorrect Q_{-1} update
 - After shifting, the old becomes the new .

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

```
IDLE Shell 3.11.8
File Edit Shell Debug Options Window Help

RESTORING DIVISION
=====
Dividend = 13 ( 1101 )
Divisor  = 3 ( 0011 )

Initial:
A = 00000
Q = 1101
M = 00011

TABLE
-----
SC   Operation                A      Q
4    Left Shift A,Q           00001    1010
     A = A - M (Negative)     11110    1010
     Restore A, Q0 = 0       00001    1010
3    Left Shift A,Q           00011    0100
     A = A - M, Q0 = 1       00000    0101
2    Left Shift A,Q           00000    1010
     A = A - M (Negative)     11101    1010
     Restore A, Q0 = 0       00000    1010
1    Left Shift A,Q           00001    0100
     A = A - M (Negative)     11110    0100
     Restore A, Q0 = 0       00001    0100

FINAL RESULT
-----
Quotient  = 4 ( 0100 )
Remainder = 1 ( 00001 )

NON-RESTORING DIVISION
=====
Dividend = 13 ( 1101 )
Divisor  = 3 ( 0011 )

Initial:
A = 00000
Q = 1101
M = 00011

TABLE
-----
SC   Operation                A      Q
4    A >= 0 : A = A - M       11110    1010
3    A < 0 : A = A + M        00000    0101
2    A >= 0 : A = A - M       11101    1010
1    A < 0 : A = A + M        11110    0100
     Final correction: A = A + M 00001    0100

FINAL RESULT
-----
Quotient  = 4 ( 0100 )
```

Bug identified: Restoring division repeatedly restores a negative partial remainder, increasing the number of operations and execution time.

Optimization: Non-restoring division avoids immediate restoration. It decides whether to add or subtract the divisor based on the sign of the current partial remainder.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

```
===== RESTART: C:/t
IEEE 754 FLOATING-POINT DEBUGGING
=====

TEST VALUES
-----
Large number = 1e+16
Small number = 1.0

SINGLE PRECISION REPRESENTATION
-----
Sign      = 0
Exponent  = 10110100
Mantissa  = 00011100001101111001010
32-bit    = 01011010000011100001101111001010

SINGLE PRECISION CALCULATION
-----
(Large + Small) - Large
Result = 0.0
STATUS: PRECISION LOSS DETECTED

DOUBLE PRECISION CALCULATION
-----
(Large + Small) - Large
Result = 0.0

DOUBLE PRECISION REPRESENTATION
-----
Sign      = 0
Exponent  = 10000110100
Mantissa  = 000111000011011110010011011111000001000000000000000
64-bit    = 0100001101000001110000110111100100110111110000010000000000000000

DEBUGGING ANALYSIS
-----
Problem : Very large and very small values have different magnitudes.
Cause   : Exponent alignment can cause significant bits of the
          smaller number to be lost.
Fix      : Use double precision or higher-precision arithmetic.

FINAL CONCLUSION
-----
Single precision may lose small values during exponent alignment.
Double precision provides more mantissa bits and better accuracy.
```

The bug occurs because floating-point numbers have limited precision. During exponent alignment, the smaller number may lose its significant bits when added to a much larger number. The optimization is to use double precision or higher-precision arithmetic when greater accuracy is required. For repeated calculations, numerically stable algorithms can also reduce rounding errors.